

range or list within square brackets. For instance, the pattern `^test/[0-9]$` will match any URL that starts with ‘test/’ followed by a single number from 0 to 9. It will not match any URL that ends with an alphabet or even a two-digit number. If we just wanted a match for the digits 2, 3, and 5, we could instead use `[235]`. For alphabets we can similarly list them or use ranges. For example `[a-q]` or `[acf]`.

If instead we wanted to match any arbitrary sequence of numbers instead of a single digit, we could use the pattern `^test/[0-9]+$`. Here the plus sign ‘+’ means one or more occurrences, so this will match any URL that is ‘test/’ followed by any one or more digit number. If you use an asterisk ‘*’ instead of the plus sign, that will include the case where there is no digit present.

If we want to be able to list articles by date — like in a blog — we need some way to match a year number, month number and day number. In this case the year number can’t be of arbitrary length, it has to be four digits. Likewise the month and day numbers have to be of two digits each. The way to specify this pattern is to use curly braces enclosing the number of times the pattern needs to repeat within them, instead of a plus or asterisk.

So to match a year you would use: `^archive/[0-9]{4}/$`. This would match any URL starting with ‘archive/’ followed by a four-digit number, and ending with a ‘/’. Likewise to match a year and month we would use: `^archive/[0-9]{4}/[0-9]{2}/$`

We can repeat the second bit that matches the month number once again to have it match days as well. If we have a situation where we want to match between, say, 3 and 6 digits, the pattern to use is `[0-9]{3,6}`.

Finally let’s look at how to match for slugs that are used for articles, categories etc. But first let’s understand what a slug is.

Often times titles for articles and other pieces of content can use special characters and spaces. Those aren’t great for URLs as they need to be escaped and leave you with odd looking URLs that aren’t good to look at or understand.

For instance let’s look at an article titled “This is an Article Title: Something’s Happened - Part 2” — quite the popular one as sequels go. If we include this title in the URL, it will appear as:

- `This%20is%20an%20Article%20Title%3A%20Something%27s%20Happened%20-%20Part%202`

As we said, it’s not pretty! What we do instead is that we create a cleaner version of this title that is URL friendly while still being decipherable. This is called a slug. If we set things up correctly Django will make one for us.